
3-Hour Technical Assessment

Mortgage Loan Management System (Mini Project)

Objective

Evaluate the candidate's ability to design and implement a **full-stack application** using:

- **ASP.NET Core REST API** (Backend)
- **Angular** (Frontend)
- **Mortgage domain understanding**
- **Clean coding, API integration, and basic validation**

Technology Stack (Mandatory)

Backend

- ASP.NET Core Web API (.NET 7 or 8)
- Entity Framework Core (In-Memory DB or SQL Server LocalDB)
- RESTful conventions

Frontend

- Angular (v15+ or v20+ acceptable)
- Angular Forms (Reactive or Template-Driven)
- HttpClient for API integration

Domain Context: Mortgage

A **Mortgage** is a loan taken to purchase a property.

In this assessment, students will build a **simple Mortgage Application Management system**.

Functional Scope (Minimal & Focused)

Only **1–2 domain models** are required.

Domain Models

1. MortgageApplication (Required)

Field	Type	Description
Id	int	Primary Key
ApplicantName	string	Borrower name
PropertyValue	decimal	Value of property
LoanAmount	decimal	Requested loan
InterestRate	decimal	Annual interest (%)
TenureYears	int	Loan tenure
Status	string	Pending / Approved / Rejected
CreatedDate	DateTime	Auto-generated

(Optional) 2. Customer (Bonus)

Field	Type
CustomerId	int
Name	string
Email	string
AnnualIncome	decimal

Assessment Tasks (3 Hours)

Part 1 – Backend (ASP.NET Core Web API) – 90 Minutes

Task 1: Create REST API Project

- Create an ASP.NET Core Web API project
 - Configure controllers and routing
 - Use **EF Core In-Memory DB** or **SQL Server**
-

Task 2: Implement Mortgage APIs

Required Endpoints

1. Create Mortgage Application

POST /api/mortgages

2. Get All Mortgage Applications

GET /api/mortgages

3. Get Mortgage By Id

GET /api/mortgages/{id}

4. Update Mortgage Status

PUT /api/mortgages/{id}/status

Business Rules

- $\text{LoanAmount} \leq \text{PropertyValue}$
 - Default Status = Pending
 - InterestRate must be > 0
 - TenureYears between 5 and 30
-

Expected Backend Output

- Proper HTTP status codes
- Model validation using Data Annotations
- Clean controller actions
- Simple service/repository pattern (optional but appreciated)

Part 2 – Frontend (Angular) – 75 Minutes

Task 3: Angular Application Setup

- Create Angular project
- Create components:
 - mortgage-list
 - mortgage-create

Task 4: UI Features

Mortgage Create Screen

- Form to submit:
 - Applicant Name
 - Property Value
 - Loan Amount
 - Interest Rate
 - Tenure
- Basic validation
- Submit data to API

Mortgage List Screen

- Display list of mortgage applications
- Show:
 - Applicant Name
 - Loan Amount
 - Status
- Button to **Approve / Reject**

Task 5: API Integration

- Use HttpClient
- Proper error handling
- Show success/error messages

Evaluation Criteria (Scoring Rubric)

Area	Weight
ASP.NET Core API Design	30%
REST & Validation Logic	20%
Angular UI & Forms	25%
API Integration	15%
Code Quality & Readability	10%

Submission Guidelines

Students must submit:

- Backend project folder
- Angular project folder
- README with:
 - How to run backend
 - How to run frontend
 - API base URL

Expected Outcome

A working mini Mortgage Management application where:

- Users can create mortgage applications
 - View them in Angular UI
 - Approve or reject applications
 - Backend APIs follow REST standards
-